

Стать IT-шником может каждый



Продвинутые возможности matplotlib. Часть 2



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Дополнительные возможности визуализации
- Работа с несколькими фигурами и осями
- Продвинутые возможности



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Введение

Визуализация данных, встроенная в pandas

Работа с датасетом Титаник



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



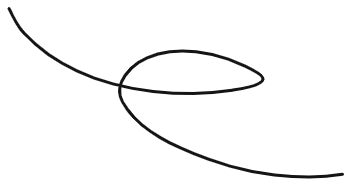
Визуализация данных, встроенная в pandas





Визуализация данных в pandas —

это простой и удобный способ создавать базовые графики непосредственно из объектов DataFrame и Series. Основана на библиотеке Matplotlib и позволяет быстро получать графическое представление данных без необходимости переходить к более специализированным инструментам визуализации.



Создание датафрейма

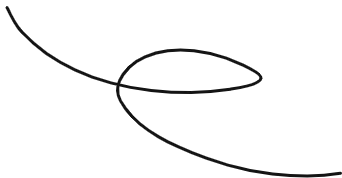
```
df = pd.DataFrame(np.random.randn(10, 4), columns=['A', 'B', 'C', 'D'])
```

Полезно знать



`.plot()` —

это метод, который поддерживает различные типы графиков через аргумент `kind`.



Типы графиков, поддерживаемых через аргумент kind

Типы	Пояснение
'line' (линии)	По умолчанию для индексированных данных
'bar' и 'barh' (вертикальные и горизонтальные столбцы)	Для сравнения значений нескольких групп
'hist' (гистограмма)	Для визуализации распределения данных
'box' (ящик с усами)	Для отображения распределения по квартилям
'scatter' (точечная диаграмма)	Для визуализации взаимосвязей между двумя числовыми переменными
'area' (площадь)	Для стековых диаграмм, показывающих суммированное значение по времени или другим переменным
'pie' (круговая диаграмма)	Для отображения долевого соотношения категорий

Метод .plot()

```
df['A'].plot(kind='hist')  
plt.show()
```

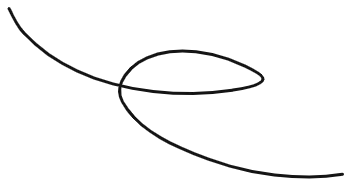
Настройка внешнего вида графиков

```
df.plot(kind='scatter', x='A', y='B', color='red', label='Group 1')  
plt.title('Scatter Plot')  
plt.xlabel('X axis')  
plt.ylabel('Y axis')  
plt.legend()  
plt.show()
```



.groupby() —

это метод, который позволяет группировать данные по одной или нескольким переменным и визуализировать агрегированные результаты.



Визуализация с использованием группировки

```
df = pd.DataFrame({'A': np.random.randint(1, 5, 10),  
                  'B': np.random.rand(10)})  
df.groupby('A').sum().plot(kind='bar')  
plt.show()
```

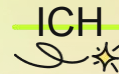
Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Задание:

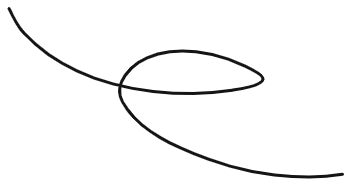
Для датафрейма из предыдущего примера постройте круговую диаграмму для средних значений при группировке по значениям столбца A.





Создание сложных макетов —

это создание макетов с несколькими графиками, pandas может быть использован в сочетании с Matplotlib, предоставляя возможность точной настройки расположения графиков.



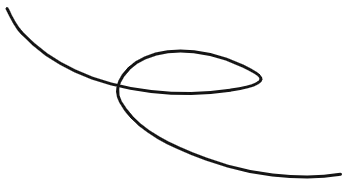
Визуализация с использованием группировки

```
fig, axes = plt.subplots(figsize=(9, 9), nrows=2, ncols=1) # эта функция объединяет plt.figure() и  
plt.subplot()  
df['A'].plot(ax=axes[0], kind='line', title='Line Plot')  
df['B'].plot(ax=axes[1], kind='bar', title='Bar Plot')  
plt.show()
```



Стековые графики —

это **графики**, которые позволяют визуализировать части целого, распределяя значения по категориям в форме стека. Полезны для сравнения пропорций во времени.



Стековые графики

```
df.plot(kind='area')  
plt.title('Stacked Area Plot')  
plt.show()
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Задание:

У вас есть три бизнес-метрики за первую половину 2023 года:

```
'sales': [3, 2, 3, 9, 10, 6],  
'signups': [5, 5, 6, 12, 14, 13],  
'visits': [20, 42, 28, 62, 81, 50]
```

Создайте дата-фрейм с этими метриками, индексами в котором будут указанные месяцы.
Постройте стековый график.



Решение:

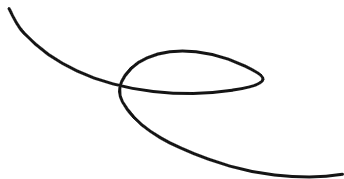
```
metrics = pd.DataFrame({
    'sales': [3, 2, 3, 9, 10, 6],
    'signups': [5, 5, 6, 12, 14, 13],
    'visits': [20, 42, 28, 62, 81, 50],
}, index=pd.date_range(start='2023/01/01', end='2023/07/01',
                        freq='ME'))

metrics.plot(kind='area')
plt.show()
```



by —

это параметр, который используется в методе `.plot()` и позволяет создавать графики с разбивкой по определенной категории, что позволяет сравнивать подгруппы данных.



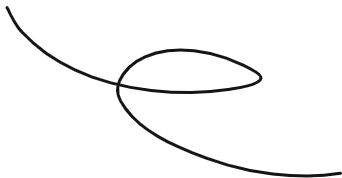
Параметр by

```
df['C'] = np.random.choice(['Category 1', 'Category 2'], size=10)
df.plot(kind="hist", figsize=(9, 9), column='B', by='C')
plt.show()
```



scatter_matrix —

это функция, которая используется для визуализации попарных отношений между несколькими переменными в DataFrame. Создает матрицу из графиков рассеяния для каждой пары переменных.



Парные графики и матрицы рассеяния

```
from pandas.plotting import scatter_matrix
df_new = pd.DataFrame({'A': np.random.normal(size=2000),
                       'B': np.random.uniform(size=2000)})
scatter_matrix(df_new, figsize=(10, 10))
plt.show()
```

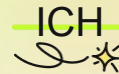
Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Задание:

Для данных о средней температуре и количестве осадков в Берлине по месяцам

```
temp = [1.8, 1.8, 4.8, 9.0, 14.0, 18.8, 19.8, 19.6, 15.0, 10.0, 5.0, 2.0]
```

```
prec = [56, 41, 53, 42, 60, 67, 81, 62, 56, 49, 48, 54]
```

создайте датафрейм и матрицу рассеяния для него.





Решение:

```
# решение
temp = [1.8, 1.8, 4.8, 9.0, 14.0, 18.8, 19.8, 19.6, 15.0, 10.0, 5.0, 2.0]
prec = [56, 41, 53, 42, 60, 67, 81, 62, 56, 49, 48, 54]
weather=pd.DataFrame({'temp': temp, 'prec': prec})
scatter_matrix(weather, figsize=(10, 10))
plt.show()
```

Стать IT-шником может каждый



Работа с датасетом Титаник



Набор данных

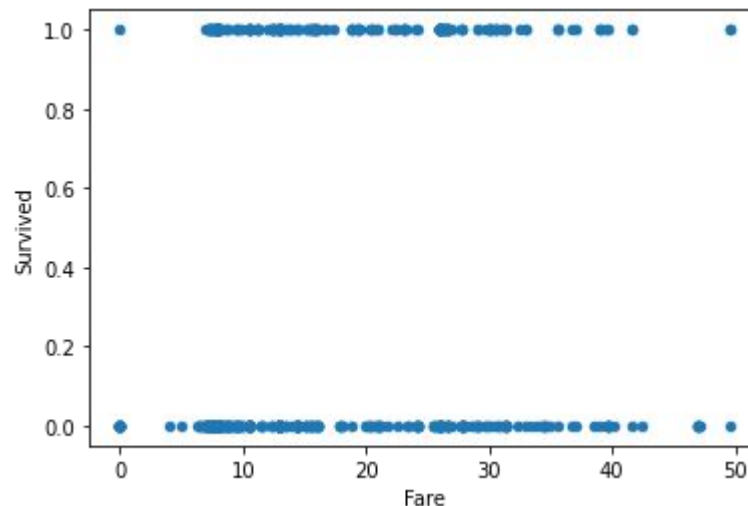
[Датасет Титаник](#)

Загрузить датасет

```
df = pd.read_csv('titanic.csv')  
df.head()
```

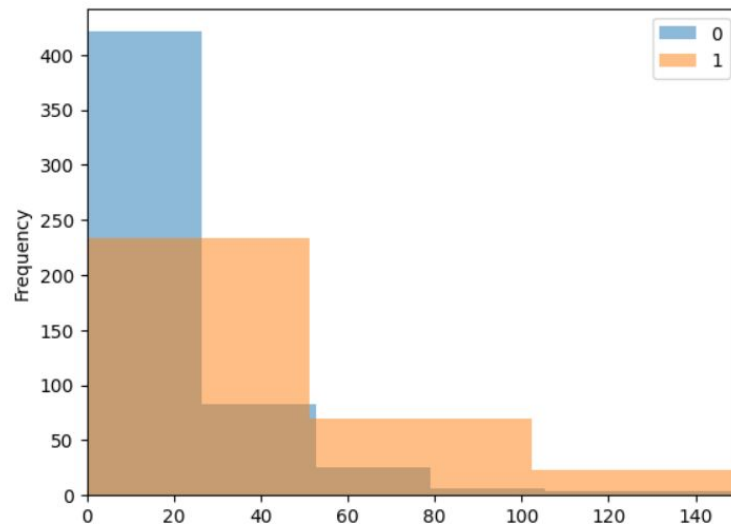
Распределение выживших и невыживших пассажиров среди тех, билет которых стоил менее 50 единиц

```
%matplotlib inline  
df[df.Fare < 50].plot.scatter(x='Fare', y='Survived')
```



Распределение выживших и невыживших в зависимости от цены билета

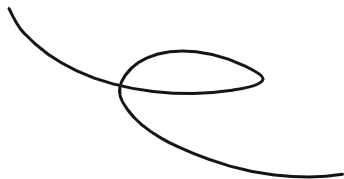
```
df.groupby('Survived').Fare.plot(kind="hist", alpha=0.5)  
plt.xlim(0, 150)  
plt.legend()  
plt.show()
```





Ядерная оценка плотности (kde) —

это способ представления распределения данных,
основанный на идее сглаживания гистограммы.



Ядерная оценка плотности (kde)

```
df.groupby('Survived').Fare.plot(kind="kde")  
plt.xlim(0, 150)  
plt.legend()  
plt.show()
```


Распределение цены

```
ax = df.Fare.plot.hist()  
ax.set_title('Visualization')  
plt.show()
```

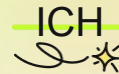
Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Задание:

В чем проявляется разница между:

```
ax = df.Fare.plot.hist()  
ax.set_title('Visualization')  
plt.show()
```

и

```
ax = df.Fare.hist()  
ax.set_title('Visualization')  
plt.show()
```

Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание 1

1. Визуализировать распределение медианных цен на дома в датасете California Housing. Этот датасет содержит информацию о различных характеристиках жилых районов Калифорнии, собранную в 1990-х годах. В нём представлены такие данные, как среднее количество комнат в доме, медианный возраст домов, среднее количество жильцов, средняя величина их дохода и медианная стоимость домов.

Практическое задание 2

2. Изучить взаимосвязь между средним количеством комнат (AveRooms) и медианной ценой на жилье (MEDV).

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ





Домашнее задание

1. Изобразите на одной системе координат графики функций $y=\sin(x)$ и $y=\log(x+1)$. Используйте две оси Y: одну для первого графика, и вторую для другого.





Домашнее задание

2. В датафрейме ниже представлены результаты соревнований: для каждого игрока указано набранное количество очков в нескольких попытках.

```
data = {  
    'Name': ['Анна', 'Антон', 'Мария', 'Макс', 'Макс', 'Алекс', 'Антон',  
            'Мария', 'Макс', 'Алекс', 'Антон', 'Анна'],  
    'Points': [145, 2524, 343, 4212, 5212, 6421, 745, 842, 524, 3224, 212,  
              1214]  
}
```

Создайте из этих данных датафрейм. Сгруппируйте данные по имени игроков. Методом `plot()` для датафреймов постройте график минимальных, максимальных и средних очков каждого игрока на одной системе координат.

Полезные ссылки

- [Официальная документация Matplotlib по продвинутым темам](#)
- [Matplotlib Python Tutorials - PythonGuides](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

